KNOWLEDGE

INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

SALEM – 637 504

DEPARTMENT

OF

INFORMATION TECHNOLOGY

LABORATORY RECORD

Course Code	::	BE23CS412
Course Name	::	EMBEDDED SYSTEMS AND IOT
Year / Sem	::	III / V
Academic Year	::	2026 – 2027 (Odd Sem)
Regulation	::	R2023

Name of the Student	::	
Register Number	::	

KNOWLEDGE INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC "A" Grade & NBA (EEE, ECE, CSE & MECH),
Approved by AICTE New Delhi, Affiliated to Anna University Chennai
NH – 47, KIOT Campus, Kakapalayam (PO), Salem – 637 504

RECORD NOTE BOOK

REGISTER NUMBER _____

Certify that this is the bonafide record of work done by Selvan / Selvi _____ of the **V Semester** _____ Branch during the year **2026 – 2027** (Odd Semester) in the **BE23CS412- Embedded Systems and IoT.**

Faculty In charge **Head of the Department**

Submitted for the End Semester Practical Examination held on _____

INFORMATION TECHNOLOGY

Vision

- ✓ To inculcate students with cutting-edge information technologies and enhance them into globally recognized engineers who are socially responsible citizen.

Mission

- ✓ To deliver reliable education with innovative techniques, software updates and programming languages to the students.
- ✓ To nurture students to work seamlessly with changing industry requirements.
- ✓ To impart skills to meet the growing demands in industry.
- ✓ To shape the students as successful professionals with resilient ethics and society consciousness.

Programme Educational Objectives (PEOs)
The graduates of Information Technology will be able to

PEO – 1	Have a successful career as an IT professional with analytical and critical thinking to meet the diversified requirements of industry, academia and research.
PEO – 2	Acquire leadership qualities with technical skills and entrepreneurship skills to solve complex engineering and social problems with ethics and environmental responsibility
PEO – 3	Pursue lifelong learning, and involve in applied research to design optimal solutions.

Programme Specific Outcome (PSOs)
The graduates of Information Technology will be able to

PSO – 1	Have proficiency in programming skills to design, develop and apply appropriate techniques, to solve complex engineering problems.
PSO – 2	Have knowledge to provide technological solutions for automation
PSO – 3	Have the skill to secure and manage large volume of data

Programme Outcomes (POs)

The graduates of Electronics and Communication Engineering will be able to:

PO – 1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO – 2	**Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and

	engineering sciences.
PO – 3	**Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO – 4	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO – 5:	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO – 6:	**The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO – 7:	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO – 8:	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO – 9:	**Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO – 10:	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO – 11:	**Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO – 12:	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Outcomes: At the end of the course, the student should be able to	
CO1	Explain the structure of embedded systems, including design processes and development tools.
CO2	Apply embedded programming concepts to build IoT solutions.
CO3	Relate embedded software components and RTOS in embedded applications.
CO4	Apply IoT architectural concepts to build IoT systems.
CO5	Apply IoT components to develop embedded IoT application.

GENERAL LABORATORY SAFETY RULES:

The Electronics & Communication (ECE) Department is committed to providing a safe laboratory environment for all students. The ECE department requires the students, staff, and faculty to adhere to basic General Laboratory Safety Rules in all department operated labs halls. Using laboratory equipment to perform any kind of experiments contains some elements of risk. Safe working habits are essential for your safety and the safety of others around you.

GENERAL LABORATORY SAFETY RULES

- Shoes shall be worn that provide full coverage of the feet and appropriate personal clothing shall be worn in laboratories.
- Students shall be familiar with the locations and operation of safety and emergency equipment such as, emergency power off, emergency telephones, and emergency exits.
- Do not displace or remove laboratory equipment without instructor or technician authorization.
- Never open or remove cover of equipment in the laboratories without instructor authorization.
- Electronic gadgets are prohibited in all ECE laboratories.

Rubric for Mark Allocation

CRITERIA	MAX. MARKS (100)	EXCELLENT (FULL MARKS)	GOOD (75%)	SATISFACTORY (50%)	NEEDS IMPROVEMENT (25%)
Preparation & Pre-lab Work	15	Fully prepared; clear understanding of objectives.	Mostly prepared with minor gaps.	Basic preparation.	Poor or no preparation.
Hardware Interfacing	20	Correct wiring and interfacing.	Minor wiring errors.	Several errors, they need help.	Incorrect or incomplete setup.
Programming & IoT Configuration	20	Error-free code and correct IoT setup.	Minor coding/ configuration errors.	Partial working code.	Code/setup fails.
Execution & Output	25	Correct output with successful communication.	Output obtained with minor issues.	Partial output.	No correct output.
Result	10	Complete observations and conclusion.	Good analysis with minor omissions.	Basic observations only.	Incomplete or missing analysis.
Viva Voce	10	Answers confidently with clear concepts.	Most answers correct.	Basic understanding.	Unable to explain concepts.

CONTENTS

S.No.	DATE	NAME OF THE EXPERIMENTS	PAGE No.	MARKS AWARDED	SIGNATURE OF THE FACULTY
CONTENT BEYOND THE SYLLABUS					
AVERAGE MARKS					

EX.NO : 01	IOT DATA ACQUISITION - READING DATA FROM A TEMPERATURE SENSOR AND DISPLAY ON LCD.
DATE :	

AIM:

To write a program and measure Temperature values using an Arduino UNO with Temperature Sensor.

SOFTWARE REQUIRED:

TINKERCAD

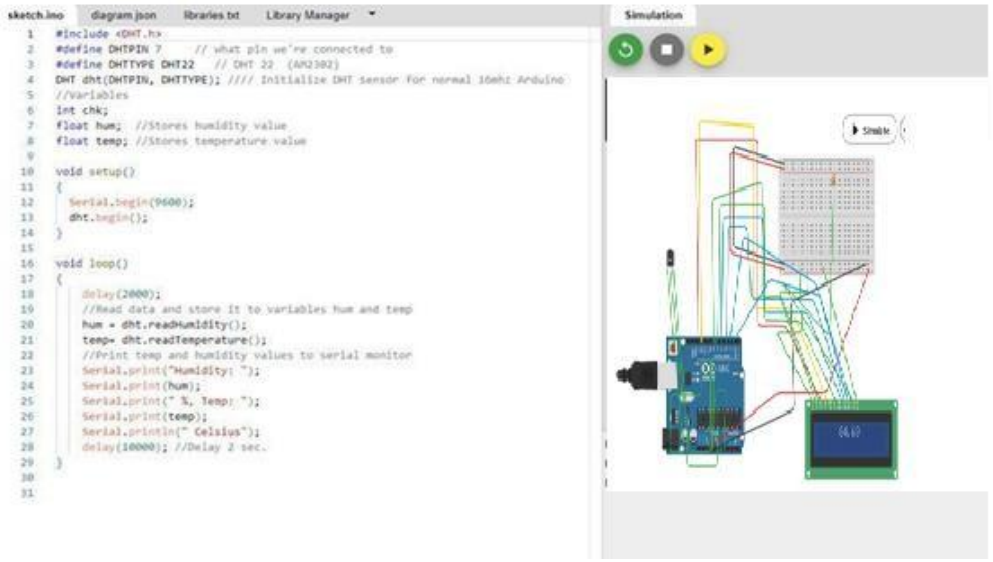
PROCEDURE:

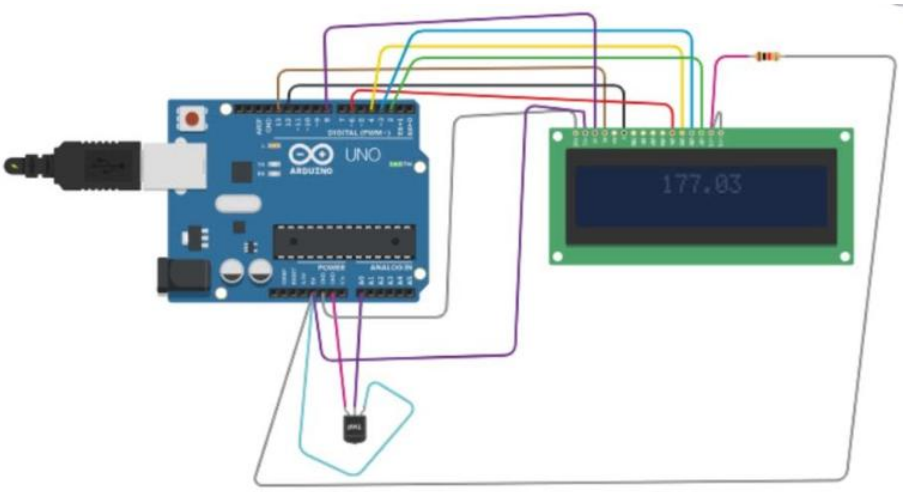
1) The DHT22 sensor Vcc pin is connected to the Arduino Uno pin of Vcc
2) The DHT22 sensor SDA pin is connected to the Arduino Uno pin of 7
3) The DHT22 sensor GND pin is connected to the Arduino Uno pin of GND
4) Power jack is connected to the Arduino Uno.
5) A USB connector is connected to Arduino Uno to monitor.
6) Connect the 12V power supply to a development board.
7) Check the output from the development board and Serial monitor

PROGRAM:

```
#include <DHT.h>
#define DHTPIN 7// what pin we're connected to
#define DHTTYPE DHT22   // DHT 22  (AM2302)
DHT dht(DHTPIN, DHTTYPE); //// Initialize DHT sensor for normal 16mhz Arduino
//Variables
int chk;
float hum;  //Stores humidity value float
    temp; //Stores temperature value void
    setup()
    {
  Serial.begin(9600);
      dht.begin();
    }
void loop()
```

SIMULATION OUTPUT:





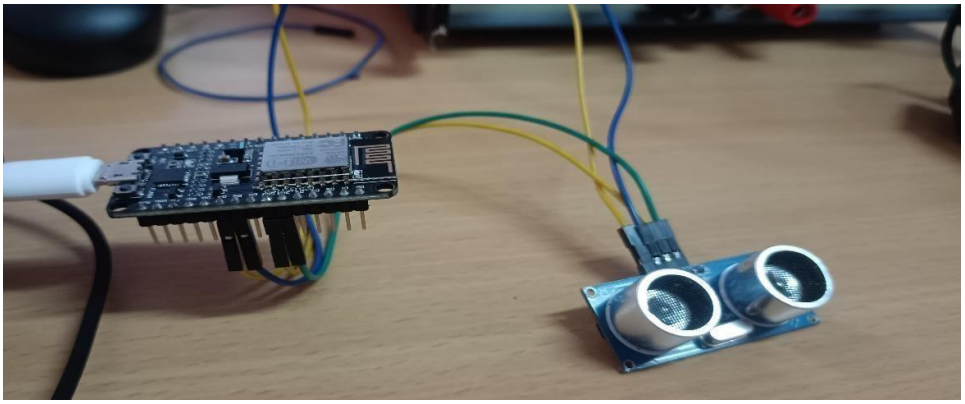
```
{
delay(2000);
//Read data and store it to variable temp
temp= dht.readTemperature();
//Print temp values to serial monitor
```

INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	

CIRCUIT CONNECTION:



EX.NO : 02	IOT DATA LOGGING AND VISUALIZATION USING IOT CLOUD PLATFORM.
DATE :	

AIM:

To write a program to measure distances using an ultrasonic sensor and send real-time distance data to Thing-Speak.

HARDWARE REQUIRED:

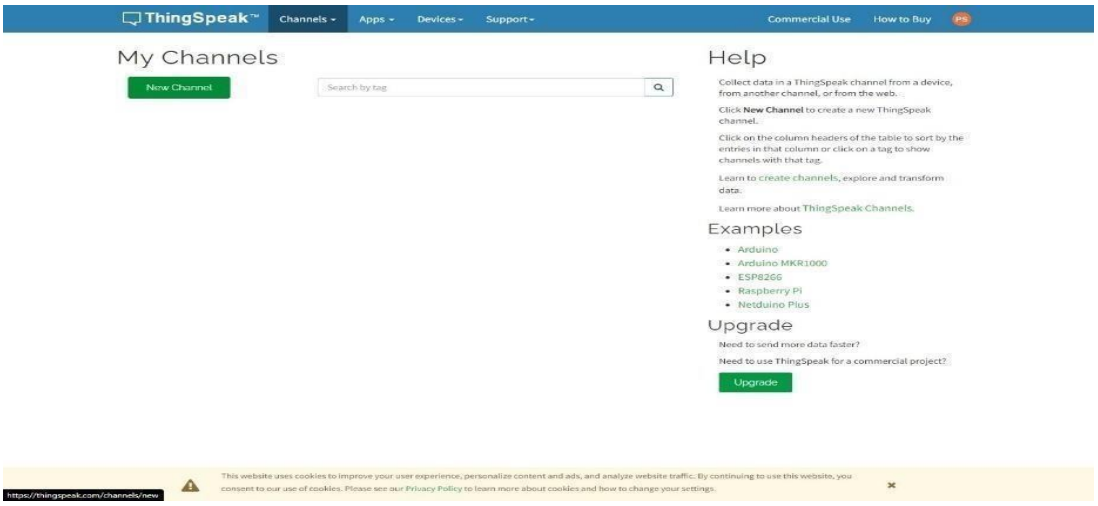
- Computer
- ESP8266 (Node MCU)
- USB Cable
- Ultrasonic Sensor (HC-SR04)
- Resistor (around 1kΩ)
- Power Supply
- Jumper Wires
- Breadboard

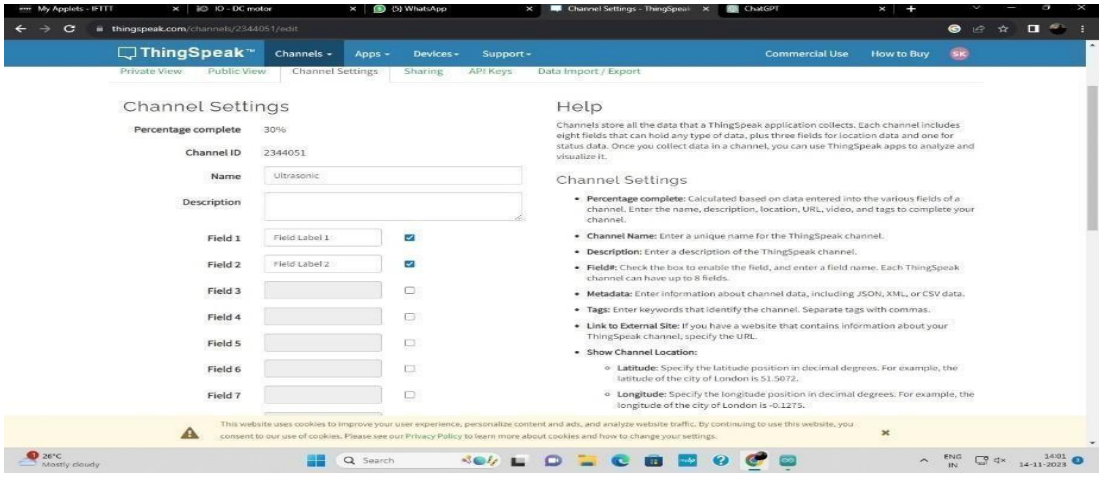
SOFTWARE REQUIRED:

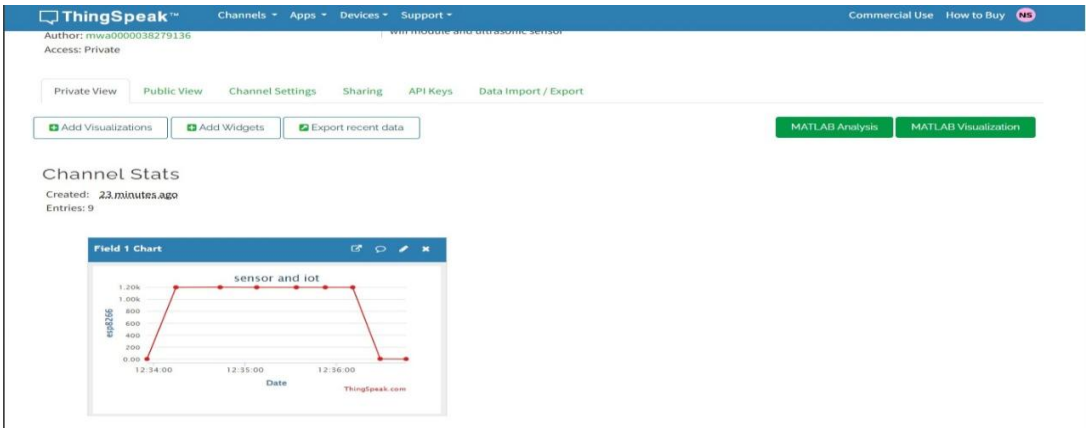
- Arduino IDE
- Thing-Speak Account

PROCEDURE:

1. **Arduino IDE**:
   - Open Arduino IDE.
   - Click on File.
   - Click on Preferences and paste the Additional Boards Manager URLs.
   - Click on Tools > Board > Boards Manager > Search ESP8266 and click Install.
   - Go to Boards, click on ESP8266, and select Node MCU 1.0 (ESP-12 E module).
   - Install the Thing-Speak library: Sketch > Include Library > Manage Libraries > Search "Thing-Speak" and click Install.
   - Write a program as per the circuit pin connections.
   - Click on Save.
   - Click on Verify.
   - Upload the code into ESP8266 using a USB cable.


ThingSpeak
Channels
Apps
Devices
Support
Commercial Use
How to Buy
My Channels
New Channel
Search by tag
Help
Collect data in a ThingSpeak channel from a device, from another channel, or from the web.
Click New Channel to create a new ThingSpeak channel.
Click on the column headers of the table to sort by the entries in that column or click on a tag to show channels with that tag.
Learn to create channels, explore and transform data.
Learn more about ThingSpeak Channels.
Examples
Arduino
Arduino MKR1000
ESP8266
Raspberry Pi
Netduino Plus
Upgrade
Need to send more data faster?
Need to use ThingSpeak for a commercial project?
Upgrade
https://thingspeak.com/channels/new
This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our Privacy Policy to learn more about cookies and how to change your settings.


ThingSpeak
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export
Channel Settings
Percentage complete 30%
Channel ID 2344051
Name Ultrasonic
Description
Field 1 Field Label 1
Field 2 Field Label 2
Field 3
Field 4
Field 5
Field 6
Field 7
Help
Channels store all the data that a ThingSpeak application collects. Each channel includes eight fields that can hold any type of data, plus three fields for location data and one for status data. Once you collect data in a channel, you can use ThingSpeak apps to analyze and visualize it.
Channel Settings
Percentage complete: Calculated based on data entered into the various fields of a channel. Enter the name, description, location, URL, video, and tags to complete your channel.
Channel Name: Enter a unique name for the ThingSpeak channel.
Description: Enter a description of the ThingSpeak channel.
Field#: Check the box to enable the field, and enter a field name. Each ThingSpeak channel can have up to 8 fields.
Metadata: Enter information about channel data, including JSON, XML, or CSV data.
Tags: Enter keywords that identify the channel. Separate tags with commas.
Link to External Site: If you have a website that contains information about your ThingSpeak channel, specify the URL.
Show Channel Location:
Latitude: Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
Longitude: Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our Privacy Policy to learn more about cookies and how to change your settings.


ThingSpeak
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Author: mwa0000038279136
Access: Private
Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export
Add Visualizations
Add Widgets
Export recent data
MATLAB Analysis
MATLAB Visualization
Channel Stats
Created: 23 minutes ago
Entries: 9
Field 1 Chart
sensor and iot
esp8266
1.20k
1.00k
800
600
400
200
0.00
12:34:00
12:35:00
12:36:00
Date
ThingSpeak.com

- Open the serial monitor and check whether the Wi-Fi is connected

2. **Thing-Speak:**
    - Create a ThingSpeak account and sign in.
    - Create a new channel with two fields: one for distance and one for timestamp.
3. **Circuit Wiring:**
    - Connect the trig pin of the ultrasonic sensor to a GPIO pin on the ESP8266 (e.g., D1).
    - Connect the echo pin of the ultrasonic sensor to a GPIO pin on the ESP8266 (e.g., D2).
    - Connect the VCC and GND of the ultrasonic sensor to the 5V and GND of the ESP8266, respectively.
    - Connect a resistor (around 1kΩ) between the echo pin and GND to create a voltage divider.

PROGRAM:

```
#include <ThingSpeak.h>
#include <ESP8266WiFi.h>
#define TRIG_PIN D1 // Change to the appropriate GPIO pin
#define ECHO_PIN D2 // Change to the appropriate GPIO pin
#define WIFI_SSID " " // WiFi Name
#define WIFI_PASS " " // Password for WiFi
#define THINGSPEAK_CHANNEL_ID 123456 // Replace with your Thing Speak channel ID
#define THINGSPEAK_API_KEY "YOUR_API_KEY" // Replace with your Thing Speak API Key
WiFiClient client;
void setup()
{ Serial.begin(115200);
 pinMode(TRIG_PIN,OUTPUT);
 pinMode(ECHO_PIN, INPUT);
ThingSpeak.begin(client);
 connectToWiFi();
  }
 void loop() {
 long duration, distance_cm;
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);
```

```
 digitalWrite(TRIG_PIN, HIGH);
 delayMicroseconds(10);
 digitalWrite(TRIG_PIN, LOW);
 duration = pulseIn(ECHO_PIN, HIGH);
  distance_cm = duration * 0.034 / 2;
  Serial.print("Distance: ");
  Serial.print(distance_cm);
  Serial.println(" cm");
  ThingSpeak.setField(1, distance_cm);
 ThingSpeak.writeFields(THINGSPEAK_CHANNEL_ID,  THINGSPEAK_API_KEY);
 delay(15000); // Send data every 15 seconds (ThingSpeak limit)

 }
 void connectToWiFi() {
 WiFi.begin(WIFI_SSID,WIFI_PASS) ;
 Serial.print("Connecting to WiFi");
 while (WiFi.status() != WL_CONNECTED) { delay(500);
 Serial.print(".");
 }
 Serial.println();
Serial.println("Connected to WiFi");
```

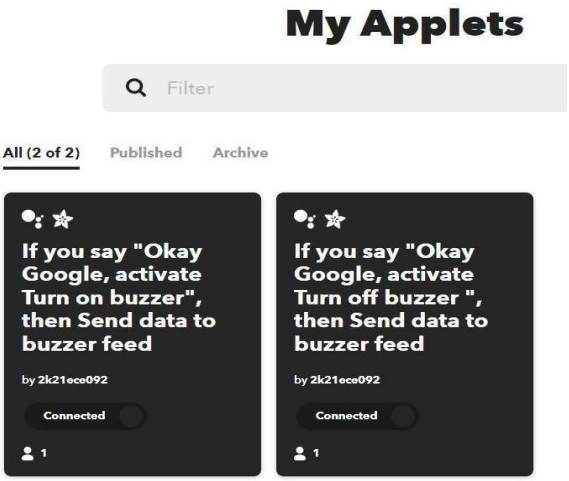
INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	

ADAFRUIT DASHBOARD:





EX.NO : 03	IMPLEMENTATION OF SIMPLE IOT BASED AUTOMATION SYSTEM
DATE :	

AIM:

To write a program to implementation of simple IoT based Automation system to control a buzzer using Google Assistant

HARDWARE REQUIRED:

- Computer
- ESP8266 (NodeMCU)
- USB Cable
- Buzzer
- Power Supply
- Jumper Wires
- Breadboard

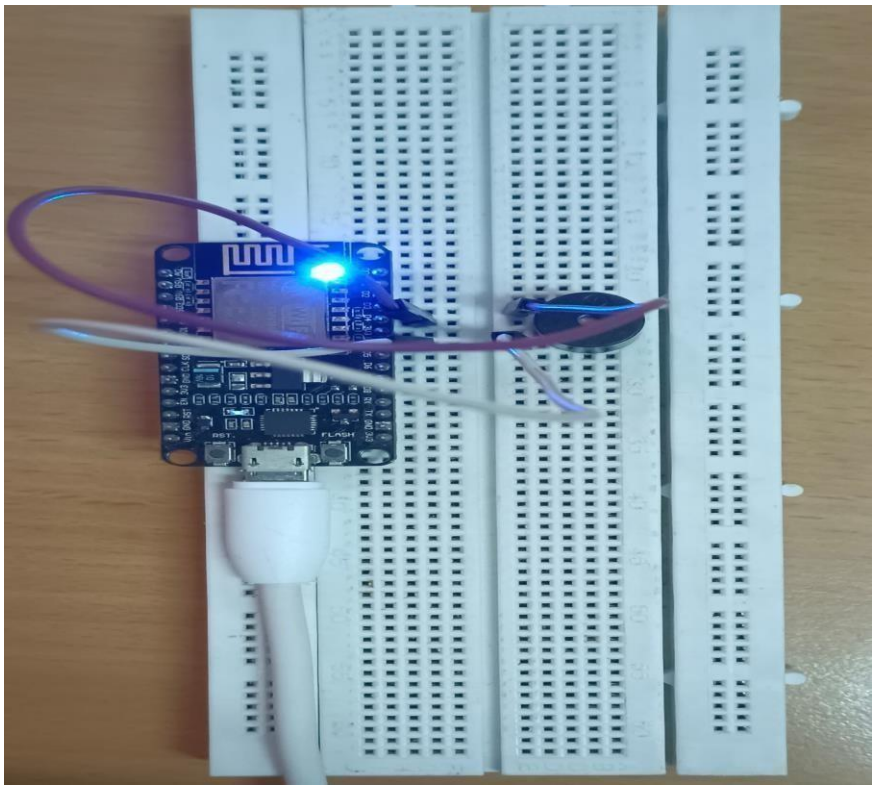
SOFTWARE REQUIRED:

- Arduino IDE
- Adafruit IO
- IFTTT
- Google Home

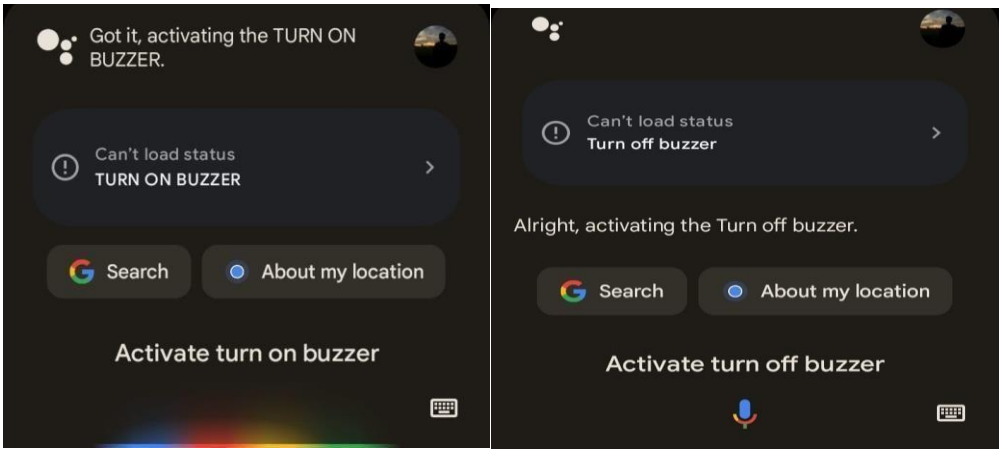
SOFTWARE PROCEDURE:

1. **Ada fruit:**
    1. Open Adafruit(adafruit.com) and Sign in
    2. Click on dashboards and Create new dashboards and give name for your experiment and click on create button
    3. Then click on your experiment and click on setting and click on Create New Block and create a block you wants,Then enter new feed name
    4. After enter feed name,Move the cursor to right side to visual Create button and click on create
    5. After create feed and click on your feed name and click on Next step
    6. And change the ON to 1 and OFF to 0,Then Click on Create block
    7. Click on new device
    8. Then now copy the API key and the User name and paste in the Program

CIRCUIT CONNECTION:



SIMULATION OUTPUT:



2. IFTTT:

1. Open IFTTT(ifttt.com) and Sigin In

Click on My Applets and Scroll down to see Create your Own and click on it

2. And Click on Add And choose a service and Search Google Assistant and click it
3. Create a activate scene and create a scene name (Eg:Turn ON DC Motor) and click on create trigger
4. Then,click on Add And choose a service and Search for Adafurit and click on it
5. Click on send data to Adafruit IO and click on Connect and Link the Adafurit to IFTTT
6. After Enter the feed name you create in Adafurit and Data to save (which means 1-ON and 0-OFF) And Click on Create Action

Then click on Continue and click on Finis

7. Click on My Applets and Scroll down to see Create your Own and click on it
8. And Click on Add And choose a service and Search Google Assistant and click on it
9. Create a activate scene and create a scene name (Eg:Turn ON DC Motor) and click on create trigger
10. Then click on Add And choose a service and Search for Ada-fruit and click on it
11. Click on send data to Ada-fruit IO and click on Connect and Link the Ada-fruit to IFTTT
12. After Enter the feed name you create in Ada-f and Data to save (which means 1-ON and 0-OFF) And Click on Create Action . Then click on Continue and click on Finis

Pin Configuration:

- Connect the positive terminal of the buzzer to a GPIO pin on the ESP8266 (e.g., GPIO2).
- Connect the negative terminal of the buzzer to the ground (GND) of the ESP8266.

Pin Configuration:

- Connect the positive terminal of the buzzer to a GPIO pin on the ESP8266 (e.g., GPIO2).
- Connect the negative terminal of the buzzer to the ground (GND) of the ESP8266.

PROGRAM:

```
#include <ESP8266WiFi.h>
#include"Adafruit_MQTT.h"
#include"Adafruit_MQTT_Client.h"
#define Buzzer D1
#define WLAN_SSID   ""
#define WLAN_PASS   ""
#define AIO_SERVER"io.adafruit.com"
#define AIO_SERVERPORT  1883
#define AIO_USERNAME  “”
#define AIO_KEY""
WiFiClient client;
Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT,
    AIO_USERNAME, AIO_KEY);
Adafruit_MQTT_Subscribe Light1 = Adafruit_MQTT_Subscribe(&mqtt,
     AIO_USERNAME"/feeds/BUZZER 01");
Void MQTT_connect();
Void setup() {
 Serial.begin(115200)
 pinMode(Buzzer, OUTPUT);
 Serial.println(); Serial.println();
 Serial.print("Connecting to ");
 Serial.println(WLAN_SSID);
 WiFi.begin(WLAN_SSID,
 WLAN_PASS);
  while (WiFi.status() !=
WL_CONNECTED)
  {
     delay(500);
   Serial.print(".");
    }
 Serial.println();
 Serial.println("WiFi connected");
 Serial.println("IP address: ");
 Serial.println(WiFi.localIP());
 mqtt.subscribe(&Light1);
   }
 void loop() {
 MQTT_connect();
 Adafruit_MQTT_Subscribe *subscription;
```

```
while ((subscription = mqtt.readSubscription(20000))) {
        if (subscription == &Light1) {
         Serial.print(F("Got: "));
          Serial.println((char *)Light1.lastread);
          int Light1_State = atoi(char  *)Light1.lastread);
          digitalWrite(Buzzer, Light1_State);
             }
             }
           }
           void
             MQTT_connect()
             { int8_t ret;
             if (mqtt.connected()) {
             return;
             }
            Serial.print("Connecting to MQTT... ");
             uint8_t retries = 3;
             while ((ret = mqtt.connect()) != 0) {
             Serial.println(mqtt.connectErrorString(ret));
             Serial.println("Retrying MQTT connection in 5 seconds...");
             mqtt.disconnect();
          delay(5000); retries-
               -;
               if (retries == 0) {
                 while (1);
               }
              }
        Serial.println("MQTT Connected!");
           }
```

INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	

EX.NO : 04	SENDING SENSOR DATA TO MOBILE APP (USING BLYNK OR MIT APP INVENTOR)
DATE :	

AIM:

To send real-time sensor data from a microcontroller (NodeMCU/ESP8266) to a mobile application using Blynk or MIT App Inventor.

COMPONENTS REQUIRED:

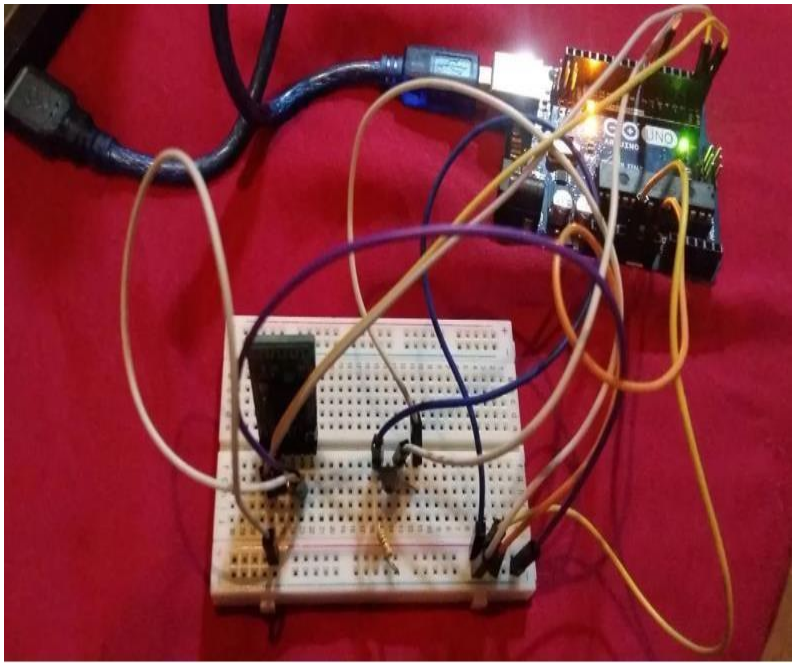
- NodeMCU (ESP8266)
- DHT11 (or Temperature) Sensor
- USB Cable
- Power Supply
- Smartphone (with Blynk or MIT App Inventor App installed)

PROCEDURE:

Circuit Wiring:

1. Connect the VCC and GND of the sensor to 3.3V and GND of NodeMCU.
2. Connect the DATA pin of DHT11 to D4(GPIO2) on NodeMCU.
3. Install the Blynk app from the Play Store/App Store and create a new project.
4. Select NodeMCU as device and WiFi as connection type.
5. Add a “Value Display” widget for temperature and humidity.
6. Note the Auth Token sent to your email.
7. In Arduino IDE, install the required libraries (Blynk, DHT).
8. Upload the code to NodeMCU.
9. Open the Blynk app and observe real-time temperature and humidity data on the mobile device.

CIRCUIT CONNECTION:



PROGRAM:

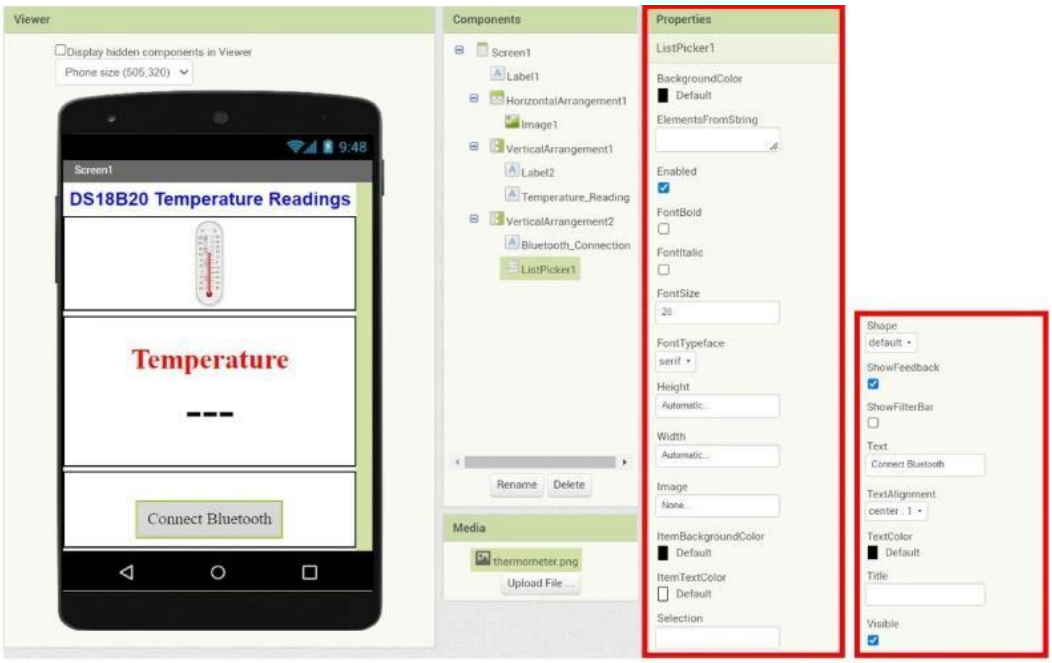
```
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
 #include <DHT.h>

char auth[] = "YourAuthToken";
char ssid[] = "YourWiFiSSID";

char pass[] = "YourWiFiPassword";
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

 void setup()
 { Serial.begin(9600);
 Blynk.begin(auth, ssid, pass);
 dht.begin();
     }
  void loop(){
  Blynk.run();
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  Blynk.virtualWrite(V5, t);
  Blynk.virtualWrite(V6, h);
  delay(2000);
     }
```

SIMULATION OUTPUT:



INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	

EX.NO : 05	SECURE COMMUNICATION IN IOT - WIFI/ BLUETOOTH INTERFACE
DATE :	

AIM:

To demonstrate secure data communication between IoT devices using WiFi or Bluetooth using Wokwi and discuss basic encryption methods.

COMPONENTS REQUIRED:

Hardware:

- NodeMCU (ESP8266)/ESP32
- Bluetooth Module (HC-05/HC-06) (optional)
- Sensor (any: DHT11/Temperature)
- USB Cable
- Power Supply

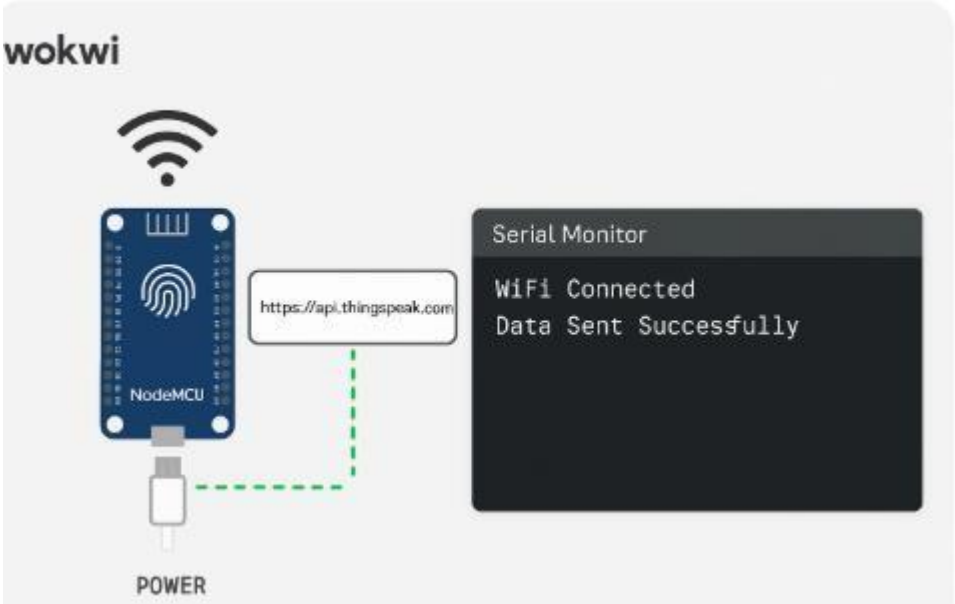
Software:

- Wokwi

Procedure: (WiFi Secure Connection with HTTPS)

1. Connect NodeMCU/ESP32 with the sensor as in previous experiments.
2. Use an endpoint that supports HTTPS (such as Wokwi or a custom HTTPS server).
3. Install and include necessary libraries to handle secure communication (WiFiClientSecure.h for HTTPS with NodeMCU).
4. (Bluetooth Secure Pairing Option)
5. Pair Bluetooth modules using default or custom PIN.
6. Restrict connections only to trusted devices.

SIMULATION OUTPUT:



PROGRAM:

```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>

const char* ssid = "YourWiFiSSID";
const char* password ="YourWiFiPassword";
const char* host = "api.thingspeak.com";

const int httpsPort = 443;

    void setup() { Serial.begin(115200);
      WiFi.begin(ssid, password);
      while (WiFi.status() != WL_CONNECTED)
        { delay(1000);
      }
  WiFiClientSecure client;
      if (!client.connect(host, httpsPort))
        {(Serial.println("Connection failed!");
       return;
      }
      {
  String url = "/update?api_key=YOUR_API_KEY&field1=24";
      client.print(String("GET ") + url + " HTTP/1.1\r\n" +
            "Host: " + host + "\r\n" +
             "Connection: close\r\n\r\n");
    }
  void loop()

}
}
```

INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	

CONTENT BEYOND SYLLABUS

EX.NO : 06	IOT-BASED SMART PARKING MANAGEMENT SYSTEM
DATE :	

AIM:

To develop an IoT-based smart parking management system that monitors parking slot availability and environmental conditions using ESP32.

SOFTWARE REQUIRED:

- WOKWI

PROCEDURE:

1. Open the Wokwi Simulator and create a new ESP32 project.
2. Add the required components: ESP32, two HC-SR04 ultrasonic sensors, DHT22 sensor, I2C LCD (16×2), servo motor, red LED, green LED, buzzer, and relay (or LED as a fan indicator).
3. Connect all the components to the ESP32 according to the specified GPIO pin connections.
4. Include the necessary libraries such as Wire, LiquidCrystal_I2C, DHT, and ESP32Servo in the Arduino program.
5. Initialize the LCD, DHT22 sensor, servo motor, ultrasonic sensors, LEDs, buzzer, and relay in the set up() function.
6. Upload the C++ program to the ESP32 in the Wokwi simulator and start the simulation.
7. The ultrasonic sensors continuously detect vehicle presence in each parking slot by measuring the distance.
8. The ESP32 calculates the number of available parking spaces based on the sensor readings.
9. Display the parking availability status on the LCD and Serial Monitor.
10. If parking spaces are available, the servo motor opens the parking gate, the green LED turns ON, and the buzzer remains OFF.
11. If all parking spaces are occupied, the LCD displays "Parking FULL", the red LED and buzzer are activated, and the servo motor closes the gate.
12. Simultaneously, the DHT22 sensor measures temperature and humidity inside the parking area.
13. When the temperature exceeds the predefined threshold (35°C), the relay (or fan indicator LED) is activated automatically.
14. Observe the changes in parking status, environmental conditions, gate operation, LEDs, buzzer, and relay by varying the sensor values in the Wokwi simulation.
15. Verify that the system successfully performs smart parking management and environmental monitoring in real time.

PROGRAM:

```
#include <LiquidCrystal.h>
#include <Servo.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

#define t1 10
#define t2 9
#define t3 8

#define IR_SENSOR_PIN 7
#define SERVO_PIN 6

#define GREEN_LED1 A0
#define GREEN_LED2 A2
#define GREEN_LED3 A4

#define RED_LED1 A1
#define RED_LED2 A3
#define RED_LED3 A5

Servo servoMotor;
int distanceThreshold = 150;

void setup() {
  lcd.begin(16,2);
  lcd.setCursor(0,0);

  pinMode(IR_SENSOR_PIN, INPUT_PULLUP);

  pinMode(GREEN_LED1, OUTPUT);
  pinMode(GREEN_LED2, OUTPUT);
  pinMode(GREEN_LED3, OUTPUT);

  pinMode(RED_LED1, OUTPUT);
  pinMode(RED_LED2, OUTPUT);
  pinMode(RED_LED3, OUTPUT);

  servoMotor.attach(SERVO_PIN);

  Serial.begin (9600);
}

long readDistance(int triggerPin, int echoPin)
{
 pinMode(triggerPin, OUTPUT);
  digitalWrite(triggerPin, LOW);
```

```
delayMicroseconds(2);
  digitalWrite(triggerPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);
  pinMode(echoPin, INPUT);
  return pulseIn(echoPin, HIGH);
}

void handleLEDs(float distance, int greenLedPin, int redLedPin) {
  if (distance >= distanceThreshold){
    digitalWrite(greenLedPin, HIGH);
    digitalWrite(redLedPin, LOW);
    Serial.println("Empty slot!!");
  } else {
    digitalWrite(redLedPin, HIGH);
    digitalWrite(greenLedPin, LOW);
    Serial.println("Car is parked");
  }
}

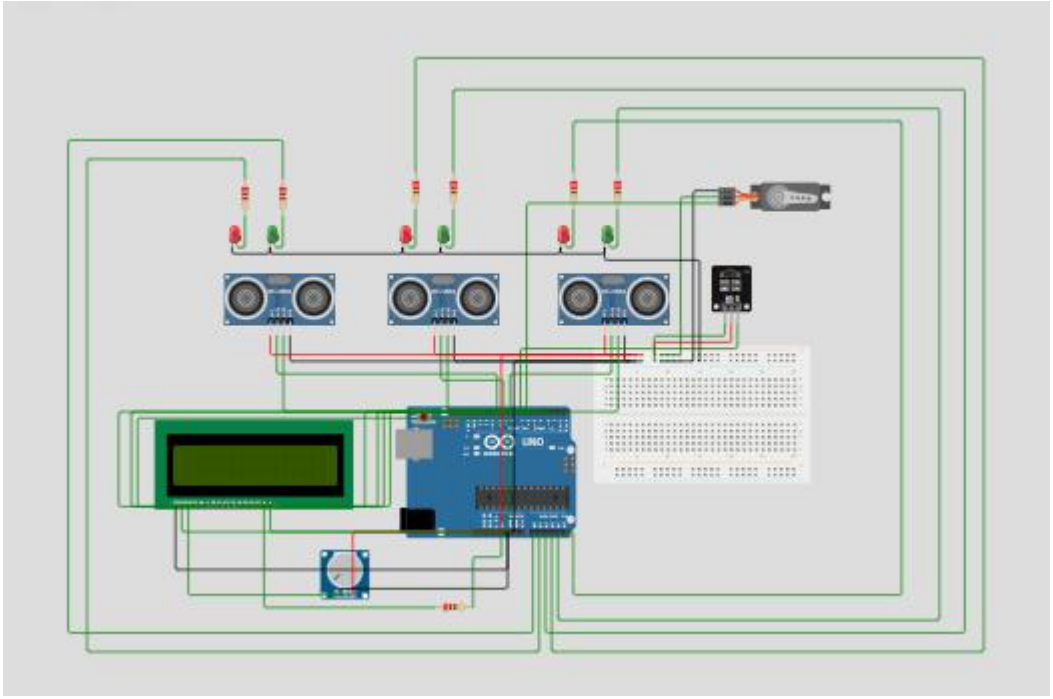
void loop() {
  //... same code as above ...
  float d1 = 0.01723 * readDistance(t1, t1);
  float d2 = 0.01723 * readDistance(t2, t2);
  float d3 = 0.01723 * readDistance(t3, t3);

  Serial.println("d1 = " + String(d1) + "cm");
  Serial.println("d2 = " + String(d2) + "cm");
  Serial.println("d3 = " + String(d3) + "cm");

  if (digitalRead(IR_SENSOR_PIN) == LOW) { // Vehicle detected by IR sensor
    if (d1 > 100 || d2 > 100 || d3 > 100) {
      servoMotor.write(90); // Assuming 90 degrees opens the gate
      delay(500);
      servoMotor.write(0); // Assuming 0 degrees closes the gate after some time
      delay(500);
    } else {
      // Do nothing or maybe sound a buzzer to indicate parking full
    }
  }

  // Handle LEDs based on the distance readings
  handleLEDs(d1, GREEN_LED1, RED_LED1);
  handleLEDs(d2, GREEN_LED2, RED_LED2);
  handleLEDs(d3, GREEN_LED3, RED_LED3);
```

SIMULATION OUTPUT:



```
d1 = 227.16cm
d2 = 402.48cm
d3 = 402.48cm
Empty slot!!
Empty slot!!
Empty slot!!
```

```
if (d1>100 & d2>100 & d3>100){
  lcd.setCursor(0,0);
  lcd.print("3 Slots Free");
  lcd.setCursor(0,1);
  lcd.print("Slot 1 2 3 Free");
  delay(500);
  }
else if((d1>100 & d2>100)|(d2>100 & d3>100)|(d3>100 & d1>100))
  {
  lcd.setCursor(0,0);
  lcd.print("2 Slots Free");
  lcd.setCursor(0,1);
    if(d1>100 & d2>100)
    lcd.print("Slot 1 & 2 Free");
    else if(d1>100 & d3>100)
    lcd.print("Slot 1 & 3 Free");
    else
    lcd.print("Slot 2 & 3 Free");
  delay(500);
  }
else if(d1<100 & d2<100 & d3<100)
  {
  lcd.setCursor(0,0);
  lcd.print("No Slot Free");
  lcd.setCursor(0,1);
  lcd.print("Parking Full");
  delay(500);
  }
else if((d1<100 & d2<100)|(d2<100 & d3<100)|(d3<100 & d1<100))
  {
  lcd.setCursor(0,0);
  lcd.print("1 Slot Free");
  lcd.setCursor(0,1);
    if(d1>100)
      lcd.print("Slot 1 is Free");
    else if (d2>100)
       lcd.print("Slot 2 is Free");
    else
       lcd.print("Slot 3 is Free");
  delay(500);
   }
  delay(100);
}
```

INFERENCE:

RESULT:

S. No.	Criteria	Max. Marks	Marks Awarded
1	Preparation & Pre-lab Work	15	
2	Software/Hardware implementation	20	
3	Programming & IoT Configuration	20	
4	Execution & Output	25	
5	Result	10	
6	Viva Voce	10	
Total		**100**	